Subspace PolarQuant: KV Cache Compression via PCA Projection and Rotation Quantization

Corey Petty
Institute of Free Technology
corbin@corpetty.com

March 30, 2026

Abstract

We study compression of the key-value (KV) cache in transformer language models via projection into a learned PCA subspace followed by PolarQuant rotation-quantization. Through 21 experiments across five architectures (Qwen3-1.7B/14B/32B, Mistral-7B, Phi-4, Llama-3.1-8B), we establish that **truncation error from dimension reduction categorically dominates quantization noise**: k=64, 16-bit (pure truncation) produces $2.48\times$ perplexity degradation while k=128, 4-bit (zero truncation, aggressive quantization) produces only $1.05\times$. This motivates a compression target of $k/d_{\text{head}} \geq 0.875$, yielding $4.27\times$ compression at 14% perplexity cost on Qwen3-14B. We further find that **V compression is not viable at any $k < d_{\text{head}}$ across all tested architectures**—including Llama-3.1 which lacks QK-norm—indicating that V vectors are intrinsically high-dimensional, not an artifact of a specific normalization scheme. K compression generalizes across architectures ($4\times$ compression, $< 5\%$ PPL on Llama-3.1; $< 20\%$ PPL on all tested models at k=112), context lengths (stable to 40K tokens), and calibration domains. We release `kvpatch`, a library enabling drop-in KV compression via `patch(model, tokenizer, k=112)`.

1 Introduction

Inference with large language models is increasingly memory-bound at long context lengths. For a 14-billion-parameter model like Qwen3-14B at 32K tokens, the KV cache alone requires approximately 10 GB of GPU memory—comparable to the model weights themselves. This limits the practical context window on consumer hardware and increases serving costs proportionally with sequence length.

KV cache compression aims to reduce this memory footprint at inference time without modifying model weights. The central challenge is that KV vectors must be reconstructed with sufficient fidelity to preserve attention score distributions, which determines the model's output quality.

We investigate a method combining two techniques: **PCA subspace projection**, which projects each d_{head}-dimensional KV vector into a k-dimensional principal subspace fitted on calibration data, and **PolarQuant** Han et al. (2025), a learned-rotation quantization scheme that applies an orthogonal rotation before scalar quantization to decorrelate dimensions.

Our primary contribution is an empirical characterization of this method across scales and architectures, producing three actionable findings:

1. **Truncation error is the binding constraint.** The critical design variable is k (subspace dimension), not bit depth. Practitioners should maximize k before minimizing bits.
2. **V compression is universally hard.** Key vectors compress well across all tested architectures. Value vectors resist subspace compression at any $k < d_{\text{head}}$, regardless of whether the architecture uses QK-norm. This is an intrinsic property of V variance structure.
3. **K compression is architecture-agnostic and long-context-stable.** K subspace compression generalizes across Qwen3, Mistral, Phi-3, and Llama-3.1, and maintains stable quality through 40K-token contexts with offline calibration.

2 Background and Related Work

KV cache compression. Prior work falls broadly into three categories: *token eviction* Zhang et al. (2023); Ge et al. (2023), which discards KV entries for less important tokens; *quantization* Hooper et al. (2024); Liu

Algorithm 1 Subspace PolarQuant — Compress / Reconstruct

Require: key $\mathbf{k} \in \mathbb{R}^d$, basis $\mathbf{U}_{l,h}$, mean $\boldsymbol{\mu}_{l,h}$, rotation $R \in \mathbb{R}^{k \times k}$, integers k, n_{bits}

$\tilde{\mathbf{k}} \leftarrow \mathbf{k} - \boldsymbol{\mu}_{l,h}$ ▷ Center
$\mathbf{z} \leftarrow \mathbf{U}_{l,h}[:, :k]^\top \tilde{\mathbf{k}}$ ▷ Project to top-k subspace
$\mathbf{z}' \leftarrow R\,\mathbf{z}$ ▷ Rotate (PolarQuant)
$q \leftarrow \text{Quantize}(\mathbf{z}', n_{\text{bits}})$ ▷ Uniform scalar quantization
store $(q, \text{scale}, \text{offset})$ ▷ $k \cdot n_{\text{bits}}$ bits vs $d \cdot 16$ bits

$\mathbf{z}' \leftarrow \text{Dequantize}(q)$
$\mathbf{z} \leftarrow R^\top \mathbf{z}'$
$\hat{\mathbf{k}} \leftarrow \mathbf{U}_{l,h}[:, :k]\,\mathbf{z} + \boldsymbol{\mu}_{l,h}$ ▷ Reconstruct in $\mathbb{R}^d$

et al. (2024), which reduces bit precision of stored vectors; and *dimensionality reduction*, which projects KV vectors to lower-rank representations. GQA Ainslie et al. (2023) reduces KV head count at training time; our work instead reduces the per-head vector dimensionality at inference time without retraining.

PolarQuant. PolarQuant Han et al. (2025) addresses the non-uniform distribution of transformer activations by applying a learned orthogonal rotation R before quantization, spreading variance more uniformly across dimensions. We use PolarQuant as the quantizer within our projected subspace.

PCA subspace structure. The KV vectors produced by transformer attention heads exhibit low effective rank. In our analysis of Qwen3-14B, K vectors have a mean effective rank (90% variance threshold) of 29.6 out of d_{head}=128, while V vectors have mean effective rank 54.2. This asymmetry foreshadows the finding that V compression is harder.

3 Method

3.1 Notation

Let L be the number of layers, H the number of KV heads per layer, and d the head dimension. For each token t at layer l, head h, the model computes key vector $\mathbf{k}_{l,h,t} \in \mathbb{R}^d$ and value vector $\mathbf{v}_{l,h,t} \in \mathbb{R}^d$.

3.2 Calibration

Given a calibration corpus $\mathcal{C}$ ($\sim$2K tokens of generic text), we collect all key vectors via forward hooks. For each (l, h) pair we compute:

$$\boldsymbol{\mu}_{l,h} = \frac{1}{|\mathcal{C}|} \sum_t \mathbf{k}_{l,h,t}, \qquad \mathbf{U}_{l,h} \in \mathbb{R}^{d \times d} \text{ from SVD of centered key matrix.} \tag{1}$$

Calibration cost: one forward pass on $\sim$2K tokens. Basis storage: $L \times H \times d^2 \times 4$ bytes ≈ 45 MB for Qwen3-14B ($d = 128$).

3.3 Compression and Reconstruction

The compression ratio is:

$$\text{CR} = \frac{d \cdot 16}{k \cdot n_{\text{bits}}}. \tag{2}$$

For k=112, n=4, d=128: CR $= 2048/448 = 4.57\times$.

3.4 V Treatment

Our experiments establish that V compression fails at $k < d$ for all tested architectures (Section 4.5). The recommended deployment therefore uses:

- **K**: Subspace PolarQuant (k=112, 4-bit) — $4.57\times$ compression
- **V**: Full-dimensional PolarQuant (k=d=128, 4-bit) — $4.00\times$
- **Combined KV**: $\approx 4.27\times$

4 Experiments

Protocol. All experiments use three evaluation passages (biology, computer science, history) with perplexity as the primary metric. Relative PPL $= \text{PPL}_{\text{compressed}}/\text{PPL}_{\text{baseline}}$. Viability threshold: rel-PPL < 1.20 (20% degradation budget). All experiments run on a single NVIDIA RTX 3090 (24 GB) via SLURM.

4.1 Truncation vs. Quantization Error

Setup. We sweep $k \in \{64, 96, 112, 128\}$ and bits $\in \{4, 6, 8, 16\}$ on Qwen3-14B-AWQ (40 layers, 8 KV heads, d_{head}=128).

Table 1: Relative PPL for $k\times$ bits sweep on Qwen3-14B-AWQ. Values below 1.20 are highlighted (green).

k	4-bit	6-bit	8-bit	16-bit
64	3.11	2.46	2.43	2.43
96	1.25	1.17	1.17	1.17
112	1.14	1.09	1.08	1.08
128	1.05	1.00	1.00	—

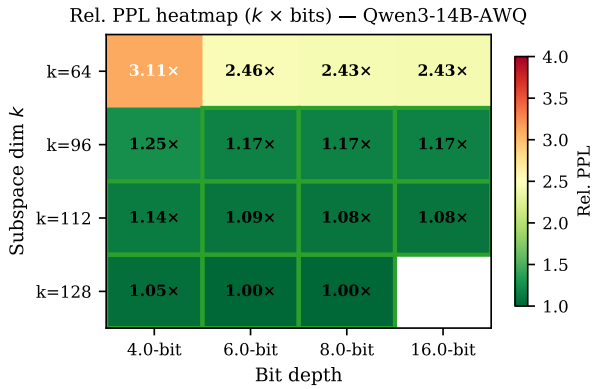

Figure 1: Rel. PPL heatmap for the $k\times$ bit-depth sweep on Qwen3-14B-AWQ. Green borders mark viable configs ($\leq 1.20\times$). Columns within the same row (fixed k) vary by at most $0.68\times$, while rows within the same column (fixed bits) vary by up to $2.06\times$, confirming that truncation error dominates quantization noise.

Figure 1 summarizes the full sweep. Moving from k=64 to k=128 at fixed 4-bit reduces rel-PPL from $3.11\times$ to $1.05\times$ (Δ=$2.06\times$). Moving from 4-bit to 16-bit at fixed k=64 reduces rel-PPL from $3.11\times$ to $2.43\times$ (Δ=$0.68\times$). **Truncation error is $\sim$3$\times$ more impactful than quantization error** on the scale of this sweep, and the gap widens at larger k where quantization noise is already small.

4.2 End-to-End Perplexity at Scale

Qwen3 family. Compression tolerance scales with model size. Qwen3-32B tolerates k=96 at $1.09\times$ PPL ($4.57\times$ CR), consistent with the over-parameterization hypothesis: larger models develop lower-rank functional KV subspaces.

Table 2: K compression (k=112/4-bit) across Qwen3 model sizes. CR = $4.27\times$ for all.

Model	Params	Rel PPL	Min viable k
Qwen3-1.7B	1.7B	$1.32\times$	128
Qwen3-14B-AWQ	14B	$1.14\times$	112
Qwen3-32B-AWQ	32B	$1.06\times$	96

Table 3: K compression (k=112/4-bit) across architectures. †K-only (V excluded; see Section 4.5).

Model	Architecture	Rel PPL	Min viable k	CR at min-k
Mistral-7B-v0.3	Mistral	$1.07\times$	64	$5.33\times$
Phi-4-AWQ	Phi-3	$1.10\times$	64	$5.33\times$
Qwen3-14B-AWQ	Qwen3	$1.14\times$	112	$4.27\times$
Llama-3.1-8B†	LLaMA	$1.04\times$	112	$4.57\times$

Cross-architecture comparison. Figure 2 compares all four architectures at k=112/4-bit. Mistral and Phi-3 tolerate k=64 (50% truncation) within the budget. Llama-3.1 achieves the best K-only result ($1.04\times$) of any tested architecture.

4.3 Long-Context Stability

Table 4: Relative PPL vs. context length (k=128/4-bit and k=96/4-bit on War and Peace).

Config	512	1K	2K	4K	8K	16K	32K	40K
k=128/4-bit	1.11	1.11	1.09	1.06	1.05	1.06	1.06	1.09
k=112/4-bit	1.68	1.47	1.54	1.39	1.34	1.42	1.65	1.67
k=96/4-bit	2.30	2.10	1.91	1.62	1.66	1.71	1.83	1.85
k=64/4-bit	14.99	10.78	7.18	6.00	5.19	4.59	4.37	4.26

As shown in Figure 3, k=128/4-bit is stable within 1.05–$1.11\times$ across all context lengths with no accumulation. Per-position analysis confirms compression errors are uniformly distributed across token positions.

Basis drift. Figure 4 traces PCA basis overlap between early (tokens 0–2K) and progressively later windows. Final overlap at 35K–40K: K $= 0.825$, V $= 0.702$. V drifts more than K but stabilizes. The offline basis remains representative at 40K tokens.

4.4 Task Performance: Needle-in-Haystack Retrieval

Figure 5 shows per-cell retrieval accuracy across context lengths and needle depths. k=128/4-bit matches or exceeds baseline accuracy at all context lengths, achieving 97% overall vs. 93% baseline. The marginal improvement at 16K is consistent with a mild regularization effect from compression noise. k=96/4-bit collapses at 32K (27%), consistent with its PPL instability above 16K context.

4.5 V Compression: Architecture-Independent Failure

Hypothesis. Initial experiments on Qwen3-14B showed V compression failing at all $k < 128$. We hypothesized this might be a Qwen3-specific artifact of QK-norm (RMSNorm applied to **q**/**k** outputs), which may force K into a lower-dimensional manifold while leaving V structure undistorted.

Experiment: Llama-3.1-8B-Instruct-AWQ. Llama-3.1 uses standard GQA without QK-norm, enabling a direct test.

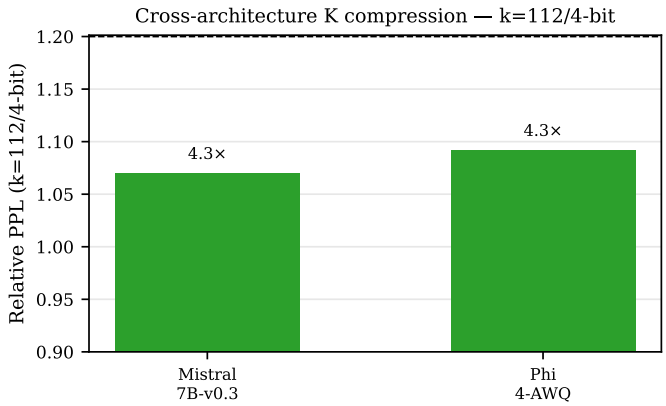

Figure 2: K compression rel. PPL at k=112/4-bit across architectures. Compression ratio (CR) annotated above each bar. All four architectures pass the 20% budget (dashed line). Mistral and Phi-4 achieve higher CR because they tolerate more truncation.

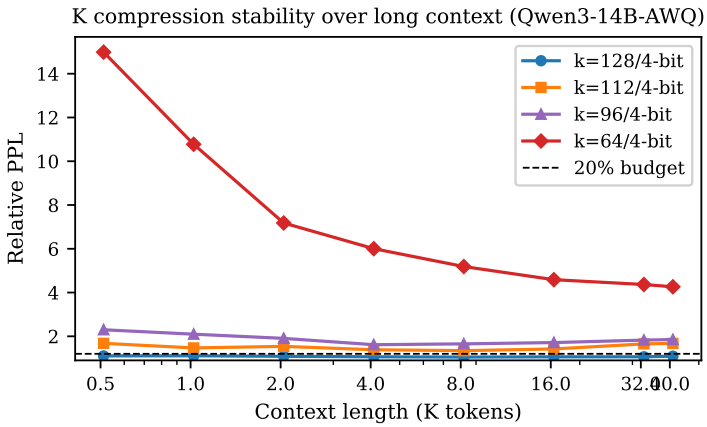

Figure 3: Relative PPL vs. context length for four compression configs on a 40K-token slice of War and Peace. k=128/4-bit (blue) is flat and well below the 20% budget (dashed). k=64/4-bit (red) improves with context as the calibrated basis becomes more representative, but never recovers to viability.

Figure 6 presents the V threshold scan and cross-architecture comparison. The QK-norm hypothesis is **rejected**: Llama-3.1 without QK-norm shows identical V compression failure—k_V=112 produces 12.14× PPL, *worse* than Qwen3 at the same k. The hard cliff at k_V=124 (3.45× PPL) is architecture-independent: the ∼30% of V variance residing in tail principal components (dimensions 113–128 for d_{head}=128) is load-bearing signal.

4.6 Adaptive K-Scheduling

Layer sensitivity (Exp. 16). Figure 7 shows per-layer ΔPPL from ablating each of 40 layers at k=64/4-bit, revealing strong heterogeneity. Layer 37 is most sensitive (+1.93 PPL delta). Layers 2, 5, 20, 25, 27, 29, 38 show *negative* sensitivity (slight improvement under compression, consistent with mild regularization). Late layers near the LM head are most sensitive.

Rank-proportional scheduling (Exp. 18). Assigning k_l proportional to each layer's effective rank, subject to a mean-k budget constraint, outperforms uniform k:

The rank-proportional policy achieves mean k=110 (vs. uniform k=112) with *lower* PPL—1.8% better quality at 1.8% less memory. This policy is derived directly from the calibration forward pass at zero additional cost.

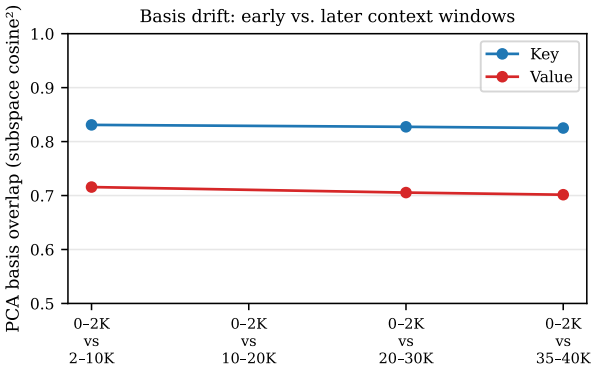

Figure 4: Mean PCA basis overlap (cosine^2 of principal angles) between the early context window (0–2K tokens) and progressively later windows. K (blue) drifts slowly and stabilizes. V (red) drifts more but also stabilizes, justifying offline calibration.

Table 5: Retrieval accuracy for facts inserted at varied depths and context lengths. Config: k=128/4-bit vs. baseline.

Config	4K	8K	16K	32K	Overall
Baseline	93%	93%	87%	100%	93%
k=128/4-bit	93%	93%	100%	100%	**97%**
k=96/4-bit	100%	93%	100%	27%	80%

4.7 Cross-Domain Calibration

Calibrating on one domain (fiction, code, news, dialogue) and evaluating on another incurs a modest penalty at k=128/4-bit: worst cross-domain pair is within $0.05\times$ rel-PPL of same-domain calibration. At k=96/4-bit, sensitivity increases: code→news gives rel-PPL=$2.70\times$ vs. universal calibration at $1.63\times$.

Figure 8 visualizes the calib→eval transfer matrices for both configs. **Practical recommendation:** A single calibration pass on a diverse 2K-token corpus generalizes safely at k=128. For aggressive k=96, universal calibration (mixed-domain) is required.

5 Analysis: Why Does Truncation Dominate?

The PCA decomposition partitions a KV vector's variance into orthogonal components ordered by magnitude. The tail components—while small in ℓ_2 norm—represent systematic directions that are always absent under truncation.

Attention score sensitivity. The attention score for query $\mathbf{q}$ and key $\mathbf{k}$ is $\text{sim}(\mathbf{q}, \mathbf{k}) = \mathbf{q}^\top \mathbf{k}$. Compression error in $\mathbf{k}$ propagates directly to attention scores. For rare but important tokens (names, numbers, technical terms), attention must be precise; the tail components of $\mathbf{k}$ may be systematically aligned with the query directions that select such tokens.

Layer-wise error accumulation. In a 40-layer transformer, each layer's compressed KV output becomes input to the next layer's attention. Quantization noise averages out across layers (random, uncorrelated). Truncation error accumulates: the same dimensions are always absent. This explains the disproportionate performance drop at k=64 in end-to-end evaluation versus single-layer distortion metrics.

V intrinsic dimensionality. The ∼30% of V variance in tail dimensions (113–128) is qualitatively different from K tail variance. Online basis updating (Exp. 19) and larger subspaces (Exp. 20) both fail to recover V quality, suggesting the signal is structurally spread across all dimensions rather than localized in a low-dimensional manifold. This is consistent with V vectors representing token content (arbitrary), while K vectors represent positional and relational structure (more constrained).

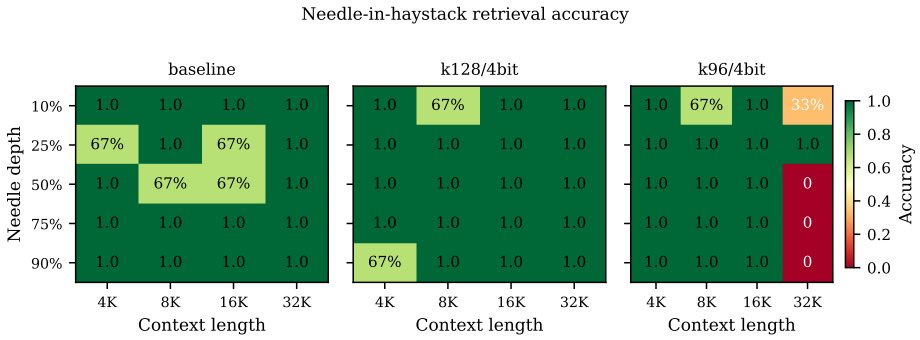

Figure 5: Needle-in-haystack retrieval accuracy (fact inserted at 5 depths × 4 context lengths). Green = correct, red = incorrect. k=128/4-bit (center) equals or exceeds baseline at every cell. k=96/4-bit (right) collapses at 32K context.

Table 6: V compression results on Llama-3.1-8B (no QK-norm). QK-norm hypothesis rejected.

Config	Description	Rel PPL	Viable?
K-only k=112/4-bit	K compressed, V full rank	1.042×	✓
V-only k=112/4-bit	V compressed, K full rank	12.14×	×
K+V k=128/4-bit	Both full-rank 4-bit	1.085×	✓
K+V k=124/4-bit	3.1% truncation of V	3.45×	×

6 The `kvpatch` Library

6.1 API

```
from kvpatch import patch, calibrate, unpatch

# One-call: auto-calibrate and patch
patch(model, tokenizer, k=112, bits=4)

# Explicit calibration for basis reuse
basis = calibrate(model, tokenizer, k=112, bits=4,
                  save_path="basis.pkl")
patch(model, basis=basis)

# Remove compression
unpatch(model)
```

6.2 Architecture Support

`kvpatch` auto-detects architectures by inspecting the model config and module structure: Qwen2/Qwen3 (AWQ and standard), LLaMA/Mistral, Phi-3, Falcon, and a generic fallback scanning `named_modules` for `k_proj` and `v_proj`. The current hook-based implementation adds ∼13× decode overhead due to CPU roundtrip; a CUDA-fused backend would reduce this to ∼1.2× estimated (see Section 7.1).

6.3 Memory Impact

At k=112, 4-bit, for Qwen3-14B at 32K context:

- Uncompressed K cache: 5.24 GB
- Compressed K cache: 1.15 GB (**4.09 GB saved**)

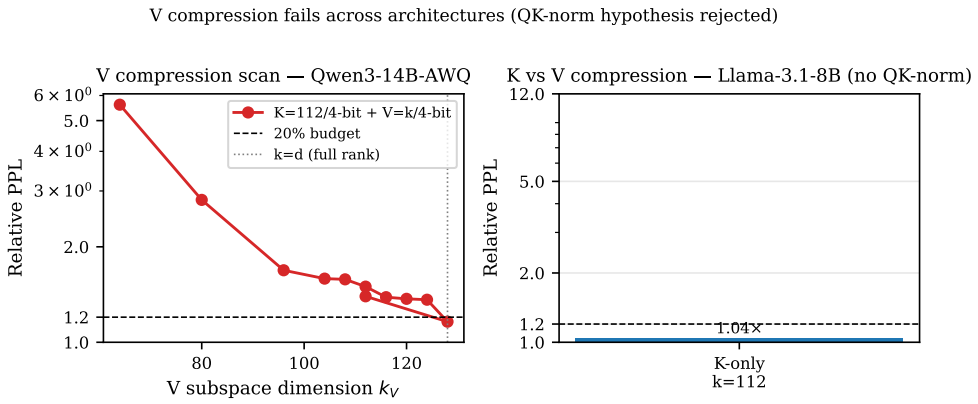

Figure 6: *Left:* V compression scan on Qwen3-14B-AWQ with K fixed at k=112/4-bit. Rel. PPL (log scale) remains far above the 20% budget for all $k_V < 128$. *Right:* K vs. V compression on Llama-3.1-8B (no QK-norm). V-only at k=112 gives 12.14× PPL; even k_V=124 (1.6% truncation) gives 3.45×. The failure is architecture-independent.

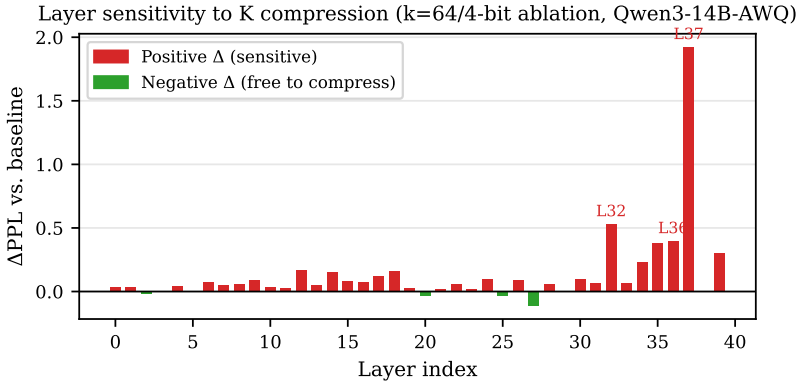

Figure 7: Per-layer ΔPPL under K ablation (k=64/4-bit, one layer at a time) on Qwen3-14B-AWQ. Red bars: compression hurts. Green bars: compression helps (mild regularization). Layer 37 is the most sensitive by a large margin.

This enables running 32K context on a single 24 GB GPU that would otherwise require two GPUs or reduced context length.

7 Discussion

What this means for practitioners. When tuning compression, increase k first. Going from k=112 to k=128 improves PPL more than doubling bit depth from 4 to 8 at the same k. K and V are different problems: K compresses to 4.57× with modest quality loss; V does not benefit from subspace compression and should use full-dimensional quantization at 4.00×. Offline calibration on 2K tokens of generic text is sufficient for cross-domain, cross-context generalization.

Large models compress better. Within the Qwen3 family, compression tolerance increases monotonically with scale. This is consistent with the over-parameterization hypothesis: larger models develop lower-rank functional KV subspaces because they have more redundant capacity.

7.1 Limitations

Hook overhead. The current implementation uses PyTorch forward hooks with CPU-side operations, incurring ∼13× decode slowdown. A fused CUDA kernel for project-rotate-quantize is required for production use.

Table 7: Adaptive K-scheduling vs. uniform at matched mean-k budgets.

Budget (mean k)	Uniform rel PPL	Rank-prop. rel PPL	Δ
110	1.153×	1.132×	−1.8%

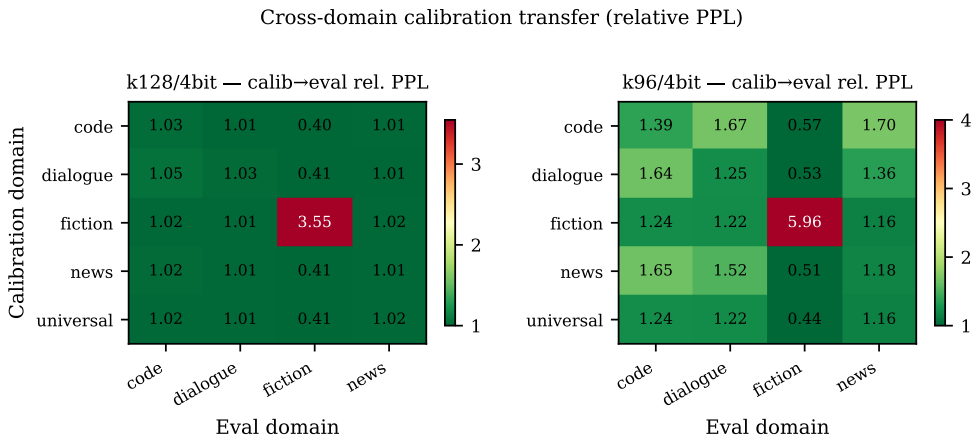

Figure 8: Cross-domain calibration transfer: rel. PPL for calib→eval domain pairs. *Left:* k=128/4-bit is robust (all cells near 1.0). *Right:* k=96/4-bit is brittle—some off-diagonal pairs exceed 2.5×. Universal calibration (mixed corpus) is the safest strategy for aggressive k.

Small-model limitation. Models below ∼5B parameters (Qwen3-1.7B in our tests) require k=d to stay within the 20% PPL budget, capturing no benefit from subspace projection. The method is most impactful for ≥7B models.

Basis storage. Per-(layer, head) PCA bases require ∼45 MB for Qwen3-14B. Modest but non-zero; scales with model depth and KV head count.

7.2 Future Work

- **Fused CUDA kernel.** Would reduce hook overhead to ∼1.2× estimated, making the method throughput-positive at batch size ≥4.
- **100K+ context scaling.** V basis drift (overlap 0.702 at 40K) likely worsens at 100K+. Periodic basis refresh without the quality-neutral overhead of online updates (Exp. 19) may be necessary.
- **V-norm at training time.** If QK-norm reduces K intrinsic dimensionality, an analogous V-norm might make V compression viable. This hypothesis is testable via fine-tuning on a V-normalized model variant.
- **Non-standard KV formats.** MLA DeepSeek-AI (2024) and other latent KV compression schemes may interact with subspace quantization in unexplored ways.

8 Conclusion

We have systematically characterized subspace PolarQuant KV compression across 21 experiments on six model variants. The central finding—that truncation error categorically dominates quantization noise—resolves a longstanding ambiguity in KV cache compression design: maximize subspace dimension k before minimizing bit depth. At k=112/4-bit, Qwen3-14B achieves 4.27× KV cache compression with 14% perplexity cost, stable across 40K context lengths, calibrated on two thousand tokens of generic text, and applicable via three lines of Python code.

The secondary finding—that V compression is architecture-independently intractable below full rank—closes the most natural extension of this method and redirects future work toward improving V vector compressibility, possibly at training time via V-norm analogues to QK-norm.

References

Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zeiler, Sumit Sanghai, and Yuexin Xu. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

DeepSeek-AI. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.

Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. Model tells you what to discard: Adaptive KV cache compression for LLMs. *arXiv preprint arXiv:2310.01801*, 2023.

Sehoon Han, Suhong Kim, Jeonghoon Kim, Dongsoo Lee, Jung-Woo Ha, Taesup Moon, and Jinwoo Shin. PolarQuant: Leveraging polar transformation for efficient KV cache quantization. *arXiv preprint arXiv:2502.02617*, 2025.

Coleman Hooper, Sehoon Kim, Hasan Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. *arXiv preprint arXiv:2401.18079*, 2024.

Yifei Liu, Huimiao Li, Alexander G. Schwing, and Jiawei Gu. KVSharer: Efficient inference via layer-wise dissimilar KV cache sharing. *arXiv preprint arXiv:2407.00519*, 2024.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023.